



Using the InterSystems ODBC Driver

Version 2026.1
2026-04-20

Using the InterSystems ODBC Driver
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1
Copyright © 2026 InterSystems Corporation
All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

1 Document Overview	1
2 Getting Started: ODBC Connections to InterSystems Databases	3
2.1 Installation Options	3
2.2 Supported ODBC Driver Managers	4
2.3 An Overview of ODBC	4
2.3.1 ODBC Connection Details	4
3 Defining an ODBC Data Source on Windows	7
3.1 Creating a DSN with the ODBC Data Source Administrator	7
3.1.1 Selecting the Correct ODBC Data Source Administrator Version	9
3.2 Using File DSNs and DSN-less Connections	10
3.2.1 ODBC Connection Strings	10
4 Defining an ODBC Data Source on UNIX®	11
4.1 Structure of the ODBC Initialization File	11
4.2 Setting up a DSN with odbcinst	13
4.3 Setting up TLS Configuration Files	13
4.4 Name and Location of the Initialization File	15
5 ODBC Installation and Validation on UNIX® Systems	17
5.1 Performing a Stand-alone Installation	17
5.2 SQL Gateway Drivers for UNIX® and Related Platforms	18
5.3 InterSystems ODBC Client Files	19
5.4 Custom Installation and Configuration for iODBC	19
5.4.1 Configuring PHP with iODBC	19
5.5 Troubleshooting for Shared Object Dependencies	21
6 ODBC Support for Python and Node.js	23
6.1 Support for pyodbc Python ODBC Bridge	23
6.1.1 Installation	23
6.1.2 macOS X Installation	24
6.1.3 Test Program	24
6.2 Support for Node.js Relational Access	25
6.2.1 Dependencies	25
6.2.2 Installation and Setup	25
6.2.3 Sample Installation and Setup on Ubuntu	26
7 Logging and Environment Variables	29
7.1 ODBC Logging on Windows	29
7.2 ODBC Logging on UNIX®	29
7.3 ODBC Environment Variables	30
8 ODBC Connection Parameters	31

List of Figures

Figure 3–1: InterSystems IRIS ODBC Data Source Setup Dialog Box	8
---	---

1

Document Overview

See the [Table of Contents](#) for a detailed listing of the subjects covered in this document.

The InterSystems ODBC driver provides a way to connect to InterSystems databases from external applications via ODBC, and allows InterSystems products to access external ODBC data sources.

This document covers the following topics:

- [Getting Started: ODBC Connections to InterSystems Databases](#) — provides an overview of the InterSystems ODBC driver.
- [Defining an ODBC Data Source on Windows](#) — describes how to use InterSystems IRIS as a ODBC data source on Windows.
- [Defining an ODBC Data Source on UNIX®](#) — describes how to use InterSystems IRIS as a ODBC data source on UNIX®.
- [ODBC Installation and Validation on UNIX® Systems](#) — describes tools to test and validate ODBC installations, and provides instructions for stand-alone installation and custom iODBC installation.
- [ODBC Support for Python and Node.js](#) — describes support for Python and Node.js open source interfaces to ODBC.
- [Logging and Environment Variables](#) — describes some tools you can use to perform troubleshooting.
- [ODBC Connection Parameters](#) — lists and describes all available ODBC connection parameters.

For more information, see the following sources:

- [Using .NET with InterSystems Software](#) describes how use the InterSystems implementation of the ADO.NET Managed Provider for relational data access and the Entity Framework for object access.
- [Using Java with InterSystems Software](#) includes information on JDBC connectivity to InterSystems IRIS from external data sources (the JDBC equivalent of what is described in this manual).
- [Using the InterSystems SQL Gateway](#) provides an overview of how the SQL Gateway works with both ODBC and JDBC.

2

Getting Started: ODBC Connections to InterSystems Databases

InterSystems provides ODBC drivers that enable you to access InterSystems databases via an ODBC connection. The InterSystems standard installation includes ODBC driver components by default. InterSystems also supports driver managers and other options as described in the following sections:

- [Installation Options](#)
- [Supported ODBC Driver Managers](#)
- [An Overview of ODBC](#)

2.1 Installation Options

To use an InterSystems database as an ODBC data source, you should first ensure that the InterSystems ODBC client driver has been installed. The following options are available:

- The InterSystems standard installation installs ODBC driver components by default (as described in the [Installation Guide](#)).
- If you perform a custom installation, you can select the `SQL client only` option to install only the ODBC client driver.
- If InterSystems IRIS is not installed on your system, you can download drivers for Windows, Linux, and macOS from the [InterSystems IRIS Drivers page](#).
- Other options are available for some UNIX® driver managers (see “[ODBC Installation and Validation on UNIX® Systems](#)”).

You must also define DSNs (Data Source Names) to provide your ODBC-aware applications with information needed to connect to InterSystems databases. Each InterSystems database can be represented by multiple DSNs, each of which can support multiple connections. See “[Defining an ODBC Data Source on Windows](#)” or “[Defining an ODBC Data Source on UNIX®](#)” for OS-specific instructions on how to perform these tasks.

Note: On Windows, InterSystems IRIS IDs use the Large Number (BigInt) datatype, so ODBC client applications must have Large Number support. For example, instances of Access 2016 previous to build 16.0.7812 will display row data as #Delet ed. This may also happen if Large Number support is not turned on in the Access Settings for the current database.

2.2 Supported ODBC Driver Managers

The InterSystems ODBC drivers are compliant with ODBC 3.5.

InterSystems ODBC supports the following ODBC driver managers:

- On Windows: the Microsoft Windows driver manager provided with the operating system.
- On UNIX®: the iODBC driver manager (for use with the Unicode and 8-bit ODBC APIs) and the unixODBC driver manager (for use with the 8-bit ODBC API). See “[ODBC Installation and Validation on UNIX® Systems](#)” for more information.

For questions about other driver managers, contact the InterSystems [WorldWide Response Center \(WRC\)](#).

InterSystems also supports Python and Node.js open source modules that work with these driver managers to provide language-specific ODBC interfaces. See the following sections for details:

- [Support for pyodbc Python ODBC Bridge](#)
- [Support for Node.js Relational Access](#)

For more complete information, including specific supported databases, see the *InterSystems Supported Platforms*.

2.3 An Overview of ODBC

An ODBC system has the following components:

- The *client application* — An application makes calls according to the Microsoft ODBC API. ODBC calls establish a connection from the client to a data source (see the section on “[ODBC Connection Details](#)”).
- The *ODBC driver manager* — The driver manager accepts calls from applications using the ODBC API and hands them off to a registered ODBC client driver. The driver manager also performs any necessary tasks so that the client application can communicate with the client driver and, ultimately, the database server.
- The *ODBC client driver* — A database-specific application that accepts calls from a client application through the ODBC driver manager and provides communication to the database server. It also performs any ODBC-related data conversions that the application requests.
- The *database server* — The actual database ultimately receiving the calls from the client application. It can be on the same or a different machine than the client driver from which it is receiving calls.
- An *initialization file* — A set of configuration information for the driver manager; depending on the operating system, it may also contain client driver information. On UNIX®, this is an actual file, frequently called `odbc.ini`. On Windows, it is a registry entry.

Note: For a particular vendor database, that vendor may offer its own version of the ODBC client driver for that platform. Oracle, for example, supplies its own ODBC driver for use with Oracle databases on Windows. This may be preferred in some cases because the vendor driver may take advantage of its knowledge of how the database works internally to optimize performance or enhance reliability.

2.3.1 ODBC Connection Details

When connecting to a database via ODBC, an application must generally provide the following connection details:

- Information about the ODBC client driver to use.
- Information on locating and accessing the database. For example, this may include the server on which the database resides and the port to use when connecting to it. The details needed depend upon the database technology.
- Login credentials to access the database, if the database is protected by a password.

In most cases, this information is stored within a DSN, which has a logical name for use within the client application. The DSN may or may not include login credentials, which can also be stored in the database initialization file, or not stored at all.

The DSNs must be registered with the ODBC driver manager.

In practice, a connection is established as follows:

1. A client application includes ODBC calls that attempt to connect to a particular DSN. A client application is linked to an ODBC driver manager, which accepts the calls.
2. The ODBC driver manager reads the initialization file to obtain the location of the ODBC client driver and load the client driver into memory.
3. Once loaded into memory, the ODBC client driver uses the ODBC initialization file to locate connection information for the DSN, as well as other information. Using this information, the client driver connects to the specified database.
4. Having established the connection, the client driver maintains communications with the database server.

CAUTION: If DSN passwords are provided, they are stored in plaintext. Hence, if a password is stored on the host machine when creating a DSN, it may be visible to other users. InterSystems does not recommend storing passwords; this should only occur if there has been a risk assessment.

3

Defining an ODBC Data Source on Windows

The following sections describe how to create a DSN for an InterSystems database on Windows, either via the Control Panel or by creating a file DSN:

- [Creating a DSN with the ODBC Data Source Administrator](#)
- [Using File DSNs and DSN-less Connections](#)

3.1 Creating a DSN with the ODBC Data Source Administrator

To create a DSN, you can use the Windows ODBC Data Source Administrator to access the InterSystems ODBC Data Source Setup dialog box:

- In the Windows Control Panel, select **Administrative Tools** and click the **ODBC Data Sources** icon (the actual icon name may vary depending on your version of Windows; see “[Selecting the Correct ODBC Data Source Administrator Version](#)” below).
- In the Windows ODBC Data Source Administrator dialog, select the **User DSN** tab and click the **Add . . .** button.
- Select **InterSystems IRIS ODBC** for the ODBC 2.5 driver or **InterSystems IRIS ODBC35** for the ODBC 3.5 driver, and click the **Finish** button.

The following illustration shows an instance of the InterSystems ODBC Data Source Setup dialog box with all required fields filled in:

Figure 3–1: InterSystems IRIS ODBC Data Source Setup Dialog Box

The fields are listed below and are required unless otherwise specified:

Data Source and Connection sections

- **Name** — Specifies the user-defined name of the DSN.
- **Description** — Optional. Provides user-defined information about the DSN.
- **Host IP Address** — Specifies the IP address to be used by the ODBC connection in dotted decimal or dotted quad form, such as “127.0.0.1”.
- **Host Port Number** — Specifies the port to be used by the ODBC connection (see “DefaultPort” in the *Configuration Parameter File Reference*).
- **Namespace** — Specifies the namespace to use as the ODBC data source.

Authentication Method section

- **Authentication Method** — Select one of the following options, depending on the security used for this database. For detailed information on these options, see *Authentication Guide*.
 - **Password** — authenticate with standard username and password.
 - **Password with SSL/TLS** — authenticate using an SSL/TLS-protected connection (see *TLS Guide*).
 - **Kerberos** — authenticate with Kerberos (see *Kerberos Authentication*). For this option, also specify the following settings:
 - **Connection Security Level** — Select Kerberos, Kerberos with Packet Integrity, or Kerberos with Encryption, as appropriate (see the Client/Server section of *About Kerberos* and the *Access Modes*).
 - **Service Principal Name** — Specify the name of the server to be used as a Kerberos principal.
- **User Name** — Optional. Specifies the username to be used by the ODBC connection. By default, this is `_SYSTEM` (not case-sensitive).

- **Password** — Optional. Specifies the password to be used by the ODBC connection. For the default username, the password is SYS (must be all upper case).

Misc section (optional settings)

- **ODBC Log** — Optional. If selected, specifies the creation of a log file of ODBC client driver activities for all InterSystems DSNs. This log is for troubleshooting; you should not turn logging on during normal operation as it will dramatically slow down ODBC performance. See “[ODBC Logging on Windows](#)” for more information.
- **Static Cursors** — Optional. If selected, enables the InterSystems ODBC client driver’s static cursor support. If this flag is off, then the cursor support provided by the ODBC Cursor Library will be used. In general, this flag should be off unless you have a specific reason for not using the ODBC Cursor Library.
- **Disable Query Timeout** — Optional. If selected, causes the ODBC client driver to ignore the value of the ODBC query timeout setting.

The ODBC query timeout setting specifies how long a client should wait for a specific operation to finish. If an operation does not finish within the specified time, it is automatically cancelled. The ODBC API provides functions to set this timeout value programmatically. Some ODBC applications, however, hard-code this value. If you are using an ODBC application that does not allow you to set the timeout value and the timeout value is too small, you can use the Disable Query Timeout option to disable timeouts.

- **Use Locale Decimal Symbol** — Optional. When selected, specifies the use of the current locale's decimal separator; not checking this sets the decimal separator in the process to a period (".") regardless of the locale. This value can have an affect when the ODBC connection is interoperating with an application that uses the decimal separator as defined for the current locale.
- **Unicode SQL Types** — Optional. This functionality is only relevant if you are working with a multibyte character set, such as in Chinese, Hebrew, Japanese, or Korean locales. If you are only using single-byte character set data, do not select this check box. If selected, this option turns on reporting of a Unicode SQL type (SQL_WVARCHAR (-9) SQLType) for string data. This allows some Microsoft applications to allocate the properly sized buffers to hold multibyte data.

If an application encounters a “SQL data type out of range” error from the Microsoft Driver Manager using **SQLBindParameter**, it can be caused by having selected this check box.

After you have created the DSN, you can use the **Test Connection** button to see if your data source is working correctly.

The **Ping** button attempts to ping the DSN host machine for the number of times specified in the **#Times** field. A popup window will display information on ping success or failure.

Tip: Windows Power Shell Commands

Windows also offers a set of Power Shell commands for manipulating DSNs from the command line. You can create a DSN in an elevated Powershell window with the **Add-OdbcDsn** command. For details, see the Power Shell documentation for Windows Data Access Components (WDAC).

3.1.1 Selecting the Correct ODBC Data Source Administrator Version

To configure user DSNs on 64-bit Windows, use the Windows Control Panel ODBC Administrator for both 32- and 64-bit programs.

To configure system DSNs for a 32-bit program, run %SystemRoot%\SysWow64\odbcad32.exe.

3.2 Using File DSNs and DSN-less Connections

Note: The `SQLDriverConnect()` function (a C call that is part of the ODBC API) can be used on all platforms to create a connection. `SQLDriverConnect()` accepts a set of name-value pairs, which are the same as those for the initialization file available on non-Windows platforms.

DSN information is typically stored in the Windows Registry (under [HKLM\SOFTWARE\ODBC]), but you can also specify connection information in a [file DSN](#) (a text file with extension .dsn).

A file DSN can be created with either the ODBC Data Source Administrator (from the `File DSN` tab) or a standard text editor. For detailed information, see the [Microsoft support site](#) (search on "file DSN").

The file DSN can specify the name of an existing DSN to use, for example:

```
[ODBC]
DSN=InterSystems ODBC Sample Code
```

or it can specify a set of key-value pairs that specify the same connection information as a standard registry entry.

File DSNs are typically stored in \Program Files\Common Files\ODBC\Data Sources, but you can use the `File DSN` tab in the ODBC Data Source Administrator to define a different default location.

3.2.1 ODBC Connection Strings

`SQLDriverConnect` takes a connection string argument that can specify connection information in three different ways:

DSN connection

Specifies the name of a regular DSN in the registry and authenticates with a username and password. For example:

```
"DSN=ODBC Samples;UID=myUsername;PWD=myPassword;"
```

To authenticate with OAuth, you can provide an access token using the `ACCESSTOKEN` parameter. For example, you can replace `UID=myUsername;PWD=myPassword;` in the above example with the following (the access token is truncated and presented as `Fgz-4mGm` for brevity):

```
"ACCESSTOKEN=Fgz-4mGm;"
```

FILEDSN connection

Specifies a [file DSN](#) rather than a registry entry. For example:

```
"FILEDSN=c:\ODBC_Samples.dsn;UID=myUsername;PWD=myPassword;"
```

DSN-less connection

Defines all connection information directly in the connection string. For example:

```
"Driver=InterSystems ODBC Driver;Server=127.0.0.1;Port=1972;Database=USER;UID=myUsername;PWD="
```

4

Defining an ODBC Data Source on UNIX®

An external application can use InterSystems databases as ODBC data sources. The following sections describe how to create a DSN for an InterSystems database on UNIX® by editing the ODBC initialization file:

- [Structure of the ODBC Initialization File](#)
- [Setting up a DSN with odbcinst](#)
- [Setting up TLS Configuration Files](#)
- [Name and Location of the Initialization File](#)

4.1 Structure of the ODBC Initialization File

The ODBC initialization file is used as follows:

- It provides information so that the driver manager can locate and connect to an available DSN, including the path of the ODBC client driver required for that particular connection.
- It defines the DSNs (and optionally includes login credentials for them). The ODBC client drivers use this information.

The following is a sample initialization file for the InterSystems ODBC driver:

```
[ODBC Data Sources]
sampleodbc=sampleodbc

[sampleodbc]
Driver           = /usr/irissys/bin/libirisodbc.so
Description      = InterSystems IRIS ODBC driver
Host             = localhost
Namespace        = USER
UID              = _SYSTEM
Password         = SYS
Port             = 1972
Protocol         = TCP
Query Timeout    = 1
Static Cursors   = 0
Trace            = off
TraceFile        = iodbctrace.log
Authentication Method = 0
Security Level   = 2
Service Principal Name = iris/localhost.domain.com

[Default]
Driver = /usr/irissys/bin/libirisodbc.so
```

This file includes the following variables:

- **ODBC Data Sources** — This section lists all DSNs defined in the file. Each entry is of the form `DSNName=SectionHeading`, where *DSNName* is the name specified by the client application and the *SectionHeading* specifies the heading under which DSN information appears in this file.
- *Driver* — Specifies the location of the client driver file to use for this DSN. In this case this is the file `libirisodbc.so`.
- *Description* — Contains an optional description of the DSN.
- *Host* — Specifies the IP address of the DSN in dotted decimal or dotted quad form, such as “127.0.0.1”.
- *Namespace* — Specifies the namespace for the DSN.
- *UID* — Specifies the username for logging in to the DSN. By default, this is `_SYSTEM` (*not* case-sensitive).
- *Password* — Specifies the password for the account specified by the *UID* entry. For default username `_SYSTEM`, the password is `SYS`. Unlike the *UID*, the password is case-sensitive.

Note: Because it is an ODBC standard to allow the storing of usernames and passwords in clear text, the sample initialization file includes the username and password required to access the sample DSN. This is meant merely as an example. A secure ODBC program prompts the user for this information and does not store it, in which case it does not appear in the initialization file at all.

- *Port* — Specifies the port for connecting to the DSN (see “DefaultPort” in the *Configuration Parameter File Reference*).
- *Protocol* — Specifies the protocol for connecting to the DSN. For InterSystems, this is always TCP.
- *Query Timeout* — If 1, causes the ODBC client driver to ignore the value of the ODBC query timeout setting.

The ODBC query timeout setting specifies how long a client should wait for a specific operation to finish. If an operation does not finish within the specified time, it is automatically cancelled. The ODBC API provides functions to set this timeout value programmatically. Some ODBC applications, however, hard-code this value. If you are using an ODBC application that does not allow you to set the timeout value and the timeout value is too small, you can use the Disable Query Timeout option to disable timeouts.

- *Static Cursors* — If 1, enables the InterSystems ODBC client driver’s static cursor support. If 0, then the cursor support provided by the ODBC Cursor Library will be used. In general, this flag should be off (that is, set to 0) unless you have a specific reason for not using the ODBC Cursor Library.
- *Trace* — Specifies whether the driver manager performs logging (“on”) or not (“off”); by default, logging is off (see “[ODBC Logging on UNIX®](#)” for more information).
- *TraceFile* — If logging is enabled by the *Trace* entry, specifies the location of the driver manager log file.
- *Authentication Method* — Specify 0 for password authentication or 1 for Kerberos.
- *Security Level* — Specify this if you use Kerberos for authentication. The allowed values are as follows:
 - 1 = Kerberos
 - 2 = Kerberos with packet integrity
 - 3 = Kerberos with encryption
- *Service Principal Name* — Specify this if you use Kerberos for authentication. This should be the name of the service principal that represents InterSystems.

For more information on Kerberos, see [Kerberos Authentication](#).

4.2 Setting up a DSN with odbcinst

A UNIX® ODBC installation includes the program odbcinst. The location is dependent on the install but may be located under `/usr/local/bin` for example.

There are two template files included with a UNIX® installation located in `install-dir\dev\odbc\redist\unixodbc`. These are:

- `odbc.ini_unixODBCtemplate` — A sample DSN entry template
- `odbcinst.ini_unixODBCtemplate` — InterSystems driver template

Edit the template files to suit your configuration. To use them, you can call odbcinst in the following ways:

- To register the driver, specify flags `-i -d -f` and your odbcinst.ini file. For example:

```
odbcinst -i -d -f odbcinst.ini_unixODBCtemplate
```

- To add a local DSN, specify flags `-i -s -h -f` and your odbc.ini file. For example:

```
odbcinst -i -s -h -f odbc.ini_unixODBCtemplate
```

- To add a System DSN, specify flags `-i -s -l -f` and your odbc.ini file. For example:

```
odbcinst -i -s -l -f odbc.ini_unixODBCtemplate
```

From: `install-dir\dev\odbc\redist\unixodbc\odbcinst.ini_unixODBCtemplate`

```
[InterSystems ODBC]
UsageCount=1
Driver=/home/iris/bin/libirisodbc.so
Setup=/home/iris/bin/libirisodbc.so
SQLLevel=1
FileUsage=0
DriverODBCVer=02.10
ConnectFunctions=YYN
APILevel=1
DEBUG=1
CPOutput=<not pooled>
```

4.3 Setting up TLS Configuration Files

InterSystems IRIS provides two template files for TLS configuration. The files are located in `install-dir\dev\odbc\redist\ssl`. The directory also contains a `readme.txt` file with further information.

- `irisodbc.ini.template` — demonstrates how an odbc.ini file entry would be configured for use with a TLS connection.
- `odbcssl.ini.template` — is an example of a TLS configuration file.

See the *InterSystems TLS Guide* for detailed information on TLS.

irisodbc.ini.template

This is a sample odbc.ini file with an entry named `[SampleSSL]` that defines a TLS connection. A working file would typically be named `install-dir/mgr/irisodbc.ini`.

```
[ODBC Data Sources]
SampleSSL = SampleTLS
```

```
[SampleTLS]
Driver = /home/guest/iris/bin/libirisodbc35.so
Description = IRIS ODBC driver
Host = localhost
Namespace = SAMPLES
UID = _SYSTEM
Password = SYS
Port = 1972
Protocol = TCP
Query Timeout = 1
Static Cursors = 0
Trace = off
TraceFile = iodbctrace.log
Service Principal Name = iris/localhost.domain.com

Authentication Method = 2
Security Level = 10
SSL Server Name = SampleSSLConfig
```

In the example above, the last three lines specify the TLS connection. The values must be defined as follows:

- *Authentication Method* must be set to 2.
- *Security Level* must be set to 10.
- *SSL Server Name* must be set to the appropriate named configuration. In this example, `SampleSSLConfig` is the *SSL Server Name* defined in the following sample file, `odbcssl.ini`.

odbcssl.ini

This is a sample TLS configuration file. In order for a process to initiate a TLS connection with these values:

- The name of this file (`<path>/odbcssl.ini`) must be specified in environment variable `ISC_SSLconfigurations`.
- The process must be using a DSN that specifies `[SampleSSLConfig]` as the *SSL Server Name* (as shown in the previous example).

```
[SampleSSLConfig]
CAFile=./CA.cer
CertFile=./Client.cer
KeyFile=./Client.key
Password=MixOfAlphaNumericAndPuncChars!
KeyType=2
Protocols=28
CipherList=ALL:!aNULL:!eNULL:!EXP:!SSLv2
VerifyPeer=1
VerifyDepth=9
```

This example defines the following values:

- *CAFile* — specifies the file containing one or more certificates used to verify the server's certificate.
- *CertFile* — specifies the file containing the client's certificate.
- *KeyFile* — specifies the file containing the client's private key file.
- *Password* — is the client's private key file password, if applicable.
- *KeyType* — specifies the type of private key used by the client.
 - 1 — DSA
 - 2 — RSA (default)
 - 3 — ECDSA
- *Protocols* — specifies which versions of TLS the client can perform.
 - 1 — SSLv2

- 2 — SSLv3
- 4 — TLSv1.0
- 8 — TLSv1.1
- 16 — TLSv1.2

Protocol combinations are specified by adding individual numbers. For example, the default setting is 28 (TLSv1 + TLSv1.1 + TLSv1.2).

- *CipherList* — specifies the list of enabled ciphersuites.
- *VerifyPeer* — specifies the peer certificate verification level.
 - 0 — None (Continue even if certificate verification fails)
 - 1 — Require (Continue only if certificate verification succeeds; default)
- *VerifyDepth* — specifies the maximum number of CA certificates allowed in peer certificate chain.

See the *InterSystems TLS Guide* for detailed information on these values.

4.4 Name and Location of the Initialization File

The initialization file can have any name, but, typically, it is called `.odbc.ini` when it is located in a user's personal directory, `odbc.ini` when located in an ODBC-specific directory. The InterSystems sample is called `irisodbc.ini` and is located in the *install-dir/mgr* directory.

To locate this file, the InterSystems ODBC client driver uses the same search order as iODBC. It looks for the file in the following places, in this order:

1. The file specified by the *ODBCINI* environment variable, if this is defined. When defined, this variable specifies a path and file, such as:

```
ODBCINI=/usr/irissys/irisodbc.ini
export ODBCINI
```

2. The `.odbc.ini` file in the directory specified by the user's *\$HOME* variable, if *\$HOME* is defined and if `.odbc.ini` exists.
3. If *\$HOME* is not defined, the `.odbc.ini` file in the "home" directory specified in the `passwd` file.
4. The file specified by the system-wide *SYSODBCINI* environment variable, if this is defined. When defined, this variable specifies a path and file, such as:

```
SYSODBCINI=/usr/irissys/irisodbc.ini
export SYSODBCINI
```

5. The file `odbc.ini` file located in the default directory for building the iODBC driver manager (*/etc/*), so that the full path and file name are */etc/odbc.ini*.

To use a different `odbc.ini` file, delete or rename the InterSystems sample initialization file to allow the driver manager to search the *\$HOME* or */etc/odbc.ini* paths. For example, go to *install-dir/bin* and execute the following command:

```
mv libodbc.so libodbc.so.old
```

and then move your user-defined `odbc.ini` to *etc/odbc*, where the driver manager can find it.

5

ODBC Installation and Validation on UNIX® Systems

The following sections provide detailed information about ODBC installation and validation on UNIX® and related operating systems:

- [Performing a Stand-alone Installation](#) — installing the InterSystems ODBC client driver and supported driver manager on UNIX®.
- [InterSystems ODBC Client Files](#) — specific file names of some of important installed components.
- [SQL Gateway Drivers for UNIX® and Related Platforms](#) — describes libraries required to run the SQL Gateway with third party drivers.
- [Custom Installation and Configuration for iODBC](#) — installing and configuring the iODBC driver manager, and configuring PHP for iODBC.
- [Troubleshooting for Shared Object Dependencies](#) — how to validate dependencies on shared objects.

5.1 Performing a Stand-alone Installation

By default, a full ODBC installation is performed with a standard InterSystems installation. If you perform a custom installation (as described in the [Installation Guide](#)), you can select the “SQL client only” option to install only the client access components (ODBC client driver).

In addition, however, a stand-alone installer is provided for InterSystems ODBC. To use this installer:

1. Create the directory where you wish to install the client, such as `/usr/irisodbc/`.
2. Copy the appropriate zipped tar file into the directory that you just created.

The `./dist/ODBC/` directory contains zipped tar files with names like the following:

```
ODBC-release-code-platform.tar.gz
```

where *release-code* is a release-specific code (that varies among InterSystems versions and releases) and *platform* specifies the operating system that the ODBC client runs on.

3. Go to the directory you created and manually unpack the `.tar` file, as follows:

```
# gunzip ODBC-release-code-platform.tar.gz
# tar xvf ODBC-release-code-platform.tar
```

This creates bin and dev directories and installs a set of files.

4. Run the **ODBCinstall** program, which will be in the directory that you created. This program creates several sample scripts and configures irisodbc.ini under the mgr directory. For example:

```
# pwd
/usr/irisodbc
# ./ODBCinstall
```

Note: **Identifying the correct platform name**

In some releases, the ./dist/ODBC/ directory contains the following command to display the platform name that identifies the file you need:

```
# ./cplatname identify
```

This command is not present in releases where it is not required.

5.2 SQL Gateway Drivers for UNIX® and Related Platforms

UNIX ODBC drivers are compiled against a specific driver manager (iODBC or unixODBC). For example, the InterSystems ODBC driver comes in versions for each driver manager (see [InterSystems ODBC Client Files](#)). These drivers require support libraries that are linked against the same driver manager. InterSystems supplies several odbcgateway libraries that are suitable for different third party drivers. The <install-dir>/bin/ directory contains the following versions of the shared objects used by the InterSystems [SQL Gateway](#). This enables you to connect from InterSystems IRIS to other ODBC client drivers. These files are not installed by default if you perform a [stand-alone ODBC installation](#).

linked against iODBC driver manager

- odbcgateway.so — supports 8-bit ODBC
- odbcgatewayiw.so — supports Unicode ODBC.

linked against unixODBC driver manager

- odbcgatewayu.so — supports 8-bit ODBC.
- odbcgatewayur64.so — supports 8-bit ODBC for 64-bit unixODBC

If you are installing a third party database driver compiled with unixODBC support (for example, the MS SQL Server ODBC driver), you must back up odbcgateway.so and rename odbcgatewayur64.so to odbcgateway.so. This change must be made for every upgrade, since the upgrade will overwrite odbcgateway.so.

For more information, see “[Using an InterSystems Database as an ODBC Data Source on UNIX®](#)”.

Note: **Setting the Shared Library Path on UNIX® Systems**

When using third-party shared libraries on a UNIX® system, LD_LIBRARY_PATH must be defined by setting the InterSystems IRIS *LibPath* parameter (see “*LibPath*” in the *Configuration Parameter File Reference*). This is a security measure to prevent unprivileged users from changing the path.

5.3 InterSystems ODBC Client Files

Depending on your configuration needs, it may be useful to know the specific file names of some of the installed components. In the following lists, *install-dir* is the InterSystems installation directory (the path that `$SYSTEM.Util.InstallDirectory()` returns on your system).

ODBC driver managers

The *install-dir/bin/* directory contains the following driver managers:

- `libiodbc.so` — The iODBC driver manager, which supports both 8-bit and Unicode ODBC APIs.
- `libodbc.so` — The unixODBC driver manager, for use with the 8-bit ODBC API.

Note: ODBC on 64-bit UNIX® platforms

Between releases of the ODBC specification, various data types such as `SQLLEN` and `SQLULEN` changed from being 32-bit values to 64-bit values. While these values have always been 64-bit on iODBC, they have changed from 32-bit to 64-bit on unixODBC. As of unixODBC version 2.2.14, the default build uses 64-bit integer values. InterSystems drivers are available for both 32-bit and 64-bit versions of unixODBC.

InterSystems ODBC client drivers

InterSystems no longer distributes ODBC 2.5 client drivers, but the ODBC 3.5 versions will convert 3.5 requests to 2.5 automatically. The *install-dir/bin/* directory contains the following versions (*.so or *.sl):

iODBC-compliant drivers

- `libirisodbc35` — supports 8-bit ODBC 3.5
- `libirisodbcw35` — supports Unicode ODBC 3.5
- `libirisodbcw.dylib` — supports Unicode ODBC for MAC OS

unixODBC-compliant drivers

- `libirisodbcuw35` — supports Unicode ODBC 3.5 for unixODBC
- `libirisodbcw6435` — supports 8-bit ODBC 3.5 for 64-bit unixODBC

5.4 Custom Installation and Configuration for iODBC

If you want to build your own iODBC driver manager to operate under custom conditions, you can do so. The iODBC executable and include files are in the directory *install-dir/dev/odbc/redist/iodbc/*. You need to set `LD_LIBRARY_PATH` (`LIBPATH` on AIX®) and the include path in order to use these directories to build your applications.

If you want to customize the iODBC driver manager, you can also do that. Download the source from the iODBC Web site (www.iodbc.org) and follow the instructions.

5.4.1 Configuring PHP with iODBC

You can use InterSystems ODBC functionality in conjunction with PHP, a scripting language that allows developers to create dynamically generated pages. The process is as follows:

1. Get or have root privileges on the machine where you are performing the installation.
2. Install the iODBC driver manager. To do this:
 - a. Download the kit.
 - b. Perform a standard installation and configuration.
 - c. Configure the driver manager for use with PHP as described in the [iODBC+PHP HOWTO](#) document on the iODBC web site (www.iodbc.org).

Note that `LD_LIBRARY_PATH` (`LIBPATH` on AIX®) in the iODBC PHP example does not get set, due to security protections in the default PHP configuration. Also, copy `libiodbc.so` to `/usr/lib` and run `ldconfig` to register it without using `LD_LIBRARY_PATH`.

3. Download the PHP source kit from <https://www.php.net> and un-tar it.
4. Download the Apache HTTP server source kit from <http://httpd.apache.org/> and un-tar it.
5. Build PHP and install it.
6. Build the Apache HTTP server, install it, and start it.
7. Test PHP and the Web server using `info.php` in the Apache root directory, as specified in the Apache configuration file (often `httpd.conf`). The URL for this is `http://127.0.0.1/info.php`.
8. Copy the InterSystems-specific initialization file, `irisodbc.ini` to `/etc/odbc.ini` because this location functions better with the Apache Web server if the `$HOME` environment variable is not defined.
9. Configure and test the `libirisodbc.so` client driver file.
10. Copy the `custom.php` file (listed below) to the Apache root directory (the directory where `info.php` is located), and tailor it to your machine for the location of your InterSystems installation directory.
11. Download and install the sample.person database from <https://github.com/interSystems/Samples-Data>, or modify `custom.php` to use your preferred sample database.
12. You can then run the `custom.php` program by pointing your browser to `http://127.0.0.1/custom.php`

custom.php listing

```
<?php
putenv("LD_LIBRARY_PATH=/usr/local/lib"); //This may be blocked by php security
echo $LD_LIBRARY_PATH;
//putenv("ODBCINSTINI=/path/to/odbcinst.ini"); //this location will be determined by your driver
install.
//putenv("ODBCINI=/path/to/odbc.ini"); //odbc.ini contains your DSNs, location determined by
your driver install.
$dsn="SAMPLES";
$user="_SYSTEM";
$password="sys";

$sql="SELECT * FROM sample.person";
if ($conn_id=odbc_connect("$dsn","", "")){
    echo "connected to DSN: $dsn";
    if($result=odbc_do($conn_id, $sql)) {
        echo "executing '$sql'";
        echo "Results: ";
        odbc_result_all($result);
        echo "freeing result";
        odbc_free_result($result);
    }else{
        echo "can not execute '$sql' ";
    }
    echo "closing connection $conn_id";
    odbc_close($conn_id);
}else{
    echo "can not connect to DSN: $dsn ";
}
?>
```


5.5 Troubleshooting for Shared Object Dependencies

After installing, you should validate dependencies on other shared objects and correct any problems. The process is as follows:

1. Use the appropriate command to list the dynamic dependencies of the InterSystems ODBC driver.

For example, on Solaris and other platforms, the command is **ldd** followed by the full path to the driver you specified in your DSN:

```
# ldd install-dir/bin/libirisodbc.so
```

Specifying the wrong file may result in dependency-related errors. If no dependencies are found, you will see a message like the following:

```
libstlport_gcc.so => not found
```

2. If there are no errors, then all dependencies are valid; if there are errors, run the following commands to force the shared object loader to look in the current directory:

```
# sh
# cd install-dir/bin
# LD_LIBRARY_PATH=`pwd`: $LD_LIBRARY_PATH
# export LD_LIBRARY_PATH
```

The **sh** command starts the Bourne shell; the **cd** command changes to the appropriate directory; and the **export** command sets the path to look up shared objects.

Note that on AIX®, you would use *LIBPATH* instead of *LD_LIBRARY_PATH*.

3. Once you have added the current directory to the path, run **ldd** again and check for missing dependencies. If any shared objects cannot be found, add them to the same directory as the ODBC client driver.

6

ODBC Support for Python and Node.js

The open source modules described here provide language-specific ODBC interfaces for Python and Node.js:

- [Support for pyodbc Python ODBC Bridge](#) — pyodbc implements the Python DB-API 2.0 specification.
- [Support for Node.js Relational Access](#) — node-odbc enables ODBC database access for Node.js client applications.

6.1 Support for pyodbc Python ODBC Bridge

pyodbc is an open source Python module that implements the DB-API 2.0 specification ([PEP 249 -- Python Database API Specification v2.0](#)), leveraging ODBC to access the underlying database. InterSystems supports use of *pyodbc* as a way to access the database from Python using the relational paradigm. This module can also be used with earlier versions of InterSystems IRIS. For general information, see the [pyodbc GitHub site](#).

Note: An InterSystems-specific implementation of DB-API 2.0 (see [Using Python DB-API](#)) was introduced with InterSystems IRIS 2022.1 and is recommended for all new Python development. The *pyodbc* implementation is still supported, and may be required for client applications that connect to older versions of InterSystems IRIS.

6.1.1 Installation

There are several sites with installation information, both for Windows and for Linux and related operating systems:

- *pyodbc* GitHub site: [pyodbc Python ODBC bridge](#)
- *pyodbc* Wiki: [Wiki Home](#)
- Microsoft *pyodbc* installation: [Python SQL Driver - pyodbc](#)
- General Python documentation: [Python](#)

The installation process is simple:

- Install Python 2 or 3 (which supports Unicode) via the [Python download](#):
- From a console with Python in the path:

```
pip install pyodbc
```

6.1.2 macOS X Installation

macOS X installation is similar to UNIX® platforms (see [Python Releases for Mac OS X](#)):

- install [homebrew](#)
- install [unixODBC](#)
- run pip install:

```
pip install pyodbc --upgrade --global-option=build_ext
--global-option="-I/usr/local/include" --global-option="-L/usr/local/lib"
```

6.1.3 Test Program

The following test program demonstrates using pyodbc to access an InterSystems IRIS database. See “[Structure of the ODBC Initialization File](#)” for an example listing the connection keywords supported by the InterSystems ODBC driver.

test.py

```
import pyodbc
import time

input("Hit any key to start")

dsn = 'IRIS Samples'
server = '127.0.0.1'
database = 'USER'
username = '_SYSTEM'
password = 'SYS'
#cnxn = pyodbc.connect('DRIVER={InterSystems ODBC35};SERVER='+server+';
#   PORT='+port+'; DATABASE='+database+';UID='+username+';PWD='+ password)
cnxn = pyodbc.connect('DSN='+dsn+')
lowptr=cnxn.getinfo(127)
highptr=cnxn.getinfo(136)
#value = PyLong_FromUnsignedLongLong(lowptr)
#print("%#5.8x"% (value))

print ("Connection high pointer: ")
print (format(highptr, '02x'))
print ("Connection high pointer: ")
print ("%#5.8x"% (highptr))
print ("Connection low pointer: ")
print ("%#5.8x"% (lowptr))
cursor = cnxn.cursor()
start= time.perf_counter()

#Sample select query
cursor.execute("SELECT * from Sample.Person")
row = cursor.fetchone()
#while row:
#   print(row)
#   row = cursor.fetchone()

end= time.perf_counter()
print ("Total elapsed time: ")
print (end-start)
input("Hit any key to end")
```

The following changes avoid returning Unicode data specifically and just directly return UTF-8 data.

```
cnxn.setdecoding(pyodbc.SQL_CHAR, encoding='latin1')
cnxn.setencoding(str, encoding='latin1')
```

This uses the narrow driver, which avoids driver managers using UCS-2 or UCS-4 Unicode and the complications of providing a driver that matches how a particular driver manager was built. For other Unicode options, see the [Unicode section of the pyodbc Wiki](#).

6.2 Support for Node.js Relational Access

The *node-odbc* open source Node.js module enables ODBC database access for Node.js client applications. According to the node-odbc site (<https://github.com/wankdanker/node-odbc>), the module is intended to be “an asynchronous/synchronous interface for node.js to unixODBC and its supported drivers” but it also works in Windows with the Windows driver manager. InterSystems IRIS supports node-odbc on both platforms.

6.2.1 Dependencies

- *InterSystems ODBC driver*

This is installed by default when you install InterSystems IRIS.

- *Node.js and npm*

Make sure Node.js version 8 or later is installed. npm is typically installed with Node.js.

- *node-odbc*

The node-odbc package is available using npm, or it can be installed locally from Github. Refer to the Github node-odbc site (<https://github.com/wankdanker/node-odbc>) for more information.

The following packages are required to build node-odbc:

- *node-gyp*

node-odbc is delivered as source and is built by npm commands using node-gyp. If you use npm to install node-odbc you may also get node-gyp installed. If not, refer to the node-gyp site (<https://www.npmjs.com/package/node-gyp>) for information about how to install it.

Depending on the OS or Linux distribution, it may be necessary to install development tools that are required by node-gyp to build the node-odbc module. No attempt is made here to document the tools required or how to install them. Refer to node-gyp and node-odbc installation instructions for more information.

- *Python and related development tools*

Python is a requirement for node-gyp. At the time of this writing, node-gyp depends on Python 2.7 but that could change in the future as new versions of node-gyp become available.

- *unixODBC (Linux/UNIX only)*

The unixODBC driver manager is required to use node-odbc on Linux, and is provided as a standard part of most Linux distributions. If not already installed on your system, see the installation instructions for your distribution. It is also available for download from the unixODBC site (<http://www.unixodbc.org/>).

6.2.2 Installation and Setup

- Make sure all dependencies are installed:
 - *Node.js and npm* (<https://nodejs.org/en/download/>) — Make sure Node.js version 8 or later is installed. npm is also required and typically installed with Node.js. Decide whether to install node modules using npm locally or globally. First step for local installation is to define a project folder, go to that folder and run 'npm init' (see the example in the following section).
 - *node-gyp* (<https://www.npmjs.com/package/node-gyp>) — This package is required to build node-odbc. It makes sense to install node-gyp globally, but a local installation will work. In either case, node-gyp will also require Python 2.7.

- `node-odbc` (<https://github.com/wankdanker/node-odbc>) — Install on your system using instructions included on the linked page. This should probably be installed locally since it needs to be rebuilt for IRIS ODBC.
- Remove UNICODE support and rebuild node-odbc. Edit `./node_modules/odbc/binding.gyp` to comment out 'UNICODE' in the 'defines' array. Save the modified binding.gyp and then in the project folder execute 'npm rebuild'.
- Make sure the appropriate InterSystems ODBC DSNs are defined. On Windows, you can use the Data Source Administrator (see “[Defining an ODBC Data Source on Windows](#)”). On non-Windows platforms, define the ODBCINI environment variable to the location of the desired `odbc.ini` file (see “[Defining an ODBC Data Source on UNIX®](#)”). It is also possible to define this in JavaScript before loading the node-odbc module.

6.2.3 Sample Installation and Setup on Ubuntu

This sample assumes that Node.js and npm have been installed on your system. If you use npm to install node-odbc you may also get node-gyp installed. The node-gyp module and its dependencies are required before you can build node-odbc.

Set up a project folder with npm init

It is okay to just take the defaults for the npm `init` options.

```
~$ mkdir my_odbc
~$ cd my_odbc
~/my_odbc$ npm init
```

This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See ``npm help json`` for definitive documentation on these fields and exactly what they do.

Use ``npm install <pkg>`` afterwards to install a package and save it as a dependency in the package.json file.

```
Press ^C at any time to quit.
package name: (my_odbc)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to /home/your_home/my_odbc/package.json:
```

```
{
  "name": "my_odbc",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
```

```
Is this OK? (yes)
~/my_odbc$
```

Install node-gyp (if not already installed globally)

```
~/my_odbc$ npm ls node-gyp
my_odbc@1.0.0 /home/your_home/my_odbc
(empty)

~/my_odbc$ npm install node-gyp --save

npm notice created a lockfile as package-lock.json. You should commit this file.
npm WARN my_odbc@1.0.0 No description
npm WARN my_odbc@1.0.0 No repository field.

+ node-gyp@3.8.0
added 97 packages from 67 contributors and audited 183 packages in 6.749s
found 0 vulnerabilities

~/my_odbc$
```

Install node-odbc

```
/my_odbc$ npm install odbc

> odbc@1.4.5 install /home/your_home/my_odbc/node_modules/odbc
> node-gyp configure build

make: Entering directory '/home/your_home/my_odbc/node_modules/odbc/build'
CXX(target) Release/obj.target/odbc_bindings/src/dynodbc.o
SOLINK_MODULE(target) Release/obj.target/odbc_bindings.node
COPY Release/odbc_bindings.node
make: Leaving directory '/home/your_home/my_odbc/node_modules/odbc/build'
npm WARN my_odbc@1.0.0 No description
npm WARN my_odbc@1.0.0 No repository field.

+ odbc@1.4.5
added 4 packages from 10 contributors and audited 187 packages in 6.187s
found 0 vulnerabilities

~/my_odbc$
```

The above may generate a large number of warnings but they can be ignored so long as the package was added successfully.

Remove UNICODE support and rebuild node-odbc

Edit `./node_modules/odbc/binding.gyp` to remove UNICODE support:

```
~/my_odbc$ nano ./node_modules/odbc/binding.gyp
```

Make sure the defines section looks like this:

```
'defines' : [
# 'UNICODE'
],
```

Now rebuild node-odbc:

```
~/my_odbc$ npm rebuild
```

Again, this command may generate a number of warnings that can be ignored. Review to make sure that no errors were encountered and the new module was successfully linked.

Set up the sample program

You can use the following JavaScript code to test the ODBC connection. This step requires a running InterSystems IRIS Server and a properly defined DSN.

```
// update this line to reference the location of irisodbc.ini on your system
process.env.ODBCINI = process.env.ODBCINI || '/opt/isc/iris/inat/mgr/irisodbc.ini';

var db = require("odbc")();
```

```
let cn = 'DSN=';

if (process.platform == "win32") {
  // Windows
  cn += 'Sampleodbc;';
} else if (process.platform == "darwin") {
  // Mac OS
  cn += 'Userunixodbc;';
} else if (process.platform == "linux") {
  cn += 'Userunixodbc;';
}
console.log(cn);

db.open(cn, function (err) {
  if (err) {
    return console.log(err);
  }
  console.log('I am connected') ;

  db.query('select * from sample.person where id<3', function cb(err, data) {
    if (err) {
      console.error(err);
    } else {
      console.log(data);
    }
  });
});
db.close(function () { });
});
```

This code assumes that the Sample.Person class is defined and compiled in the namespace specified by the DSN and that it has data with ID values less than 3.

Run the sample

```
~/my_odbc$ node index.js
~/my_odbc$ node sample.js

DSN=Userunixodbc;
I am connected
[ { ID: 1,
  Age: 3,
  DOB: '2015-09-28',
  FavoriteColors: '',
  Name: 'Ulman,George L.',
  SSN: '293-31-5406',
  Spouse: 0,
  Home_City: 'Newton',
  Home_State: 'MI',
  Home_Street: '6958 Main Avenue',
  Home_Zip: '20649',
  Office_City: 'Xavier',
  Office_State: 'NY',
  Office_Street: '7313 Madison Avenue',
  Office_Zip: '73226' },
  { ID: 2,
    Age: 16,
    DOB: '2002-04-07',
    FavoriteColors: 'Green',
    Name: 'Pascal,Vincent A.',
    SSN: '973-94-3185',
    Spouse: 0,
    Home_City: 'Xavier',
    Home_State: 'ND',
    Home_Street: '3788 Madison Drive',
    Home_Zip: '80569',
    Office_City: 'Washington',
    Office_State: 'SC',
    Office_Street: '1206 Second Place',
    Office_Zip: '37389' } ]

~/my_odbc$
```


7

Logging and Environment Variables

The InterSystems ODBC driver provides various tools for debugging and diagnostics:

- [ODBC Logging on Windows](#)
- [ODBC Logging on UNIX®](#)
- [ODBC Environment Variables](#)

CAUTION: Enable logging only when you need to perform troubleshooting. You should not enable logging during normal operation, because it will dramatically slow down performance.

7.1 ODBC Logging on Windows

To enable logging for an ODBC data source on Windows, use the *ODBC Data Source Administrator* to change the logging information in the DSN (see “[Creating a DSN with the ODBC Data Source Administrator](#)” for detailed usage information). In the ODBC Data Source Administrator, make the following changes:

- To enable logging for the client driver, find the definition of the DSN that you want to log, open it, and check the box labeled `ODBC Log` (or `Log` or variations).
- To enable logging for the driver manager, click the `Tracing` tab and then click the `Start Tracing Now` button. The `Log File Path` field determines the name and location of the trace file.

Note: The default log file name is `IRISODBC.log`, and the default location is `C:\Users\Public\Logs`. You can change these values by setting the `IRISODBCTRACEFILE` environment variable (see “[ODBC Environment Variables](#)” later in this chapter).

7.2 ODBC Logging on UNIX®

On UNIX®, enable logging for ODBC as follows:

- To enable logging for the client driver, use the `IRISODBCTRACE` environment variable (as described later in “[ODBC Environment Variables](#)”). Also configure the ODBC initialization file.

- To enable logging for the driver manager, set the *Trace* entry in the ODBC initialization file (see “[Structure of the ODBC Initialization File](#)” in “[Using an InterSystems Database as an ODBC Data Source on UNIX®](#)”). In the same file, the *TraceFile* entry specifies the name of the log file to create.

Tip: If you enable logging but the log file is not updated, either you might not have privileges to write to the file or the client application may have loaded the SO before you enabled logging. In the latter case, stop and restart the client application to force it to reload the SO and get the logging flag.

7.3 ODBC Environment Variables

This section describes the environment variables that control the InterSystems ODBC client driver. Typically you use these only for debugging or diagnostics.

IRISODBCDEFTIMEOUT

This variable allows you to specify the duration of a timeout for a default login. Its value is in seconds.

IRISODBCPID

This boolean variable controls the automatic appending of the process ID number to the log file name. Set the value to 1 to enable appending, or 0 to disable. By default, appending is off.

With *IRISODBCPID* enabled, if the base log file is *IRISODBC.log* and is in your current directory, then the process ID of 21933 generates a full log file name of *IRISODBC.log.21933*.

Both *IRISODBCPID* and *IRISODBCTRACEFILE* affect the file name. For example, on Windows if you use *IRISODBCTRACEFILE* to set the base file name of the log file (for instance, to *C:/home/mylogs/mylog.txt* and enable *IRISODBCPID*, then log file names will be of the form *C:/home/mylogs/mylog.txt.21965*.

IRISODBCTRACE (UNIX® Only)

This boolean variable controls client driver logging. Set the value to 1 to enable client driver logging, or 0 to disable. For more information, see “[ODBC Logging on UNIX®](#)” earlier in this chapter.

IRISODBCTRACEFILE

This variable specifies the location and name of the log file. This can be useful for placing the log file in a unique directory or giving it a unique name. The default name for the log file is *IRISODBC.log*. The default location is as follows:

- For UNIX®, the log is generated in the current directory by default.
- For Windows, the default location for the log file is *%PUBLIC%\Logs*. This directory is accessible by all users and allows just one location for the log to be created.

IRISODBCTRACETHREADS

This variable controls whether the log also includes threading information. Set the value to 1 to enable inclusion of threading information, or 0 to disable.

It can be useful to enable this additional kind of logging, if you need to debug a threaded application. However, it adds many extra lines to the log for most ODBC applications.

8

ODBC Connection Parameters

This section lists and describes the connection property options provided by the InterSystems ODBC driver. See [Creating a DSN with the ODBC Data Source Administrator](#) (Windows) and [Structure of the ODBC Initialization File](#) (UNIX) for more information and examples.

AccessToken

An OAuth access token (used instead of username and password). For example, you could replace `UID=myUsername;PWD=myPassword;` in the connection string with:

```
"ACCESSTOKEN=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3M;"
```

The access token string in this example has been truncated for brevity. If a `AccessToken` is specified, then no username or password are used as these are mutually exclusive.

Authentication Method

Defines the security method to be used. Valid values are:

- `0` = Password — authenticate with standard username and password.
- `1` = Kerberos — authenticate with Kerberos. This option also requires settings for [Security Level](#) and [Service Principal Name](#).
 - `Connection Security Level` — Select Kerberos, Kerberos with Packet Integrity, or Kerberos with Encryption, as appropriate (see the Client/Server section of About Kerberos and the Access Modes).
 - `Service Principal Name` — Specify the name of the server to be used as a Kerberos principal.
- `2` = Password with SSL/TLS — authenticate using an SSL/TLS protected connection. [Security Level](#) must be set to `10` if this option is selected.

For detailed information on these options, see Authentication Guide.

See [Creating a DSN with the ODBC Data Source Administrator](#) (Windows) and [Structure of the ODBC Initialization File](#) (UNIX) for detailed information and examples.

Description

Provides user-defined information about the [DSN](#) or [initialization file](#).

Disable Query Timeout

If set to 1, causes the ODBC client driver to ignore the value of the [Query Timeout](#) setting. This is useful in cases where an ODBC application has a hard-coded Query Timeout value that is too short for your purposes.

Driver

This setting is implemented in different ways depending on your operating system:

- In Windows, specifies the name of the InterSystems ODBC driver (where the name is the key to a Windows registry entry). For example: "Driver={InterSystems ODBC}" or "Driver={InterSystems IRIS ODBC35}".
- In UNIX, specifies the full path to the InterSystems ODBC client driver file. For example: "Driver = /usr/irissys/bin/libirisodbc.so"

DSN

A pre-configured Data Source Name (DSN) in the ODBC Data Source Administrator.

FileDSN

The path to a file containing the same information as a DSN. For example "FILEDSN = C:\\ODBC_Samples.dsn". See [Structure of the ODBC Initialization File](#) and [Using File DSNs and DSN-less Connections](#) for more information and examples.

Host (or Server)

The hostname or IP address of the InterSystems IRIS server (for example, localhost or 127.0.0.1).

Log

When set to LOG=1, the InterSystems ODBC client driver creates a log file (default name IRISODBC.log in a specific system directory) that records its activities.

This log is for troubleshooting; you should not turn logging on during normal operation as it will dramatically slow down ODBC performance. See [ODBC Logging in Windows](#) for more information.

LogFile

Specifies the location of a log file. For example, "LOGFILE=d:/work/filelog.log". If a log file has already been specified, this will close the old file and redirect logging to the new one.

Namespace (or Database)

The InterSystems IRIS Namespace to connect to (for example, USER or SAMPLES).

Port

The Superserver port for the InterSystems IRIS instance. The default is 1972 (see "DefaultPort" in the *Configuration Parameter File Reference*).

Protocol

Specifies the protocol for connecting to the DSN. For InterSystems, this is always TCP.

PWD (or Password)

Specifies the password for the account specified by the [UID](#) parameter. For default username _SYSTEM, the password is SYS. Unlike the UID, the password is case-sensitive.

Query Timeout

Specifies the number of seconds a client should wait for a specific operation to finish. If an operation does not finish within the specified time, it is automatically cancelled. If you are using an ODBC application that has a hard-coded timeout value, Query Timeout can be disabled by setting [Disable Query Timeout](#).

Security Level

Specifies the type of connection when you use Kerberos for authentication. The allowed values are as follows:

- 1 = Kerberos
- 2 = Kerberos with packet integrity
- 3 = Kerberos with encryption
- 10 = This parameter must be set to 10 if [Authentication Method](#) is 2.

For more information on Kerberos, see Kerberos Authentication.

Service Principal Name (or SSL Server Name)

Specifies the name of the server to be used as a Kerberos principal when using Kerberos for authentication. This should be the name of the service principal that represents InterSystems. For example, "Service Principal Name = iris/localhost.domain.com"

SQLDialect (InterSystems IRIS only)

Defines the SQL dialect used when preparing and executing dynamic SQL statements. It allows InterSystems IRIS to process SQL source code intended for other database systems by overriding its default SQL behavior.

Valid values are IRIS, MSSQLSERVER, MSSQL, and SYBASE. The default is IRIS. Support for MSSQLSERVER and SYBASE dialects is limited to a subset of the TSQL grammar supported by the InterSystems IRIS TSQL language mode.

This setting is crucial for compatibility when migrating applications from other databases or when using the [SQL Gateway](#) to connect to external data sources that use different SQL dialects.

Can be set:

- within the DSN configuration using the [Windows ODBC Data Source Administrator](#) tool.
- programmatically in ObjectScript using the %SYSTEM.TSQL.SetDialect() method or by setting the %Dialect property of a %SQL.Statement instance.
- via the [InterSystems Management Portal](#) in the TSQL Compatibility settings.

Static Cursors

When set to 1, enables the InterSystems ODBC client driver's static cursor support. If 0, then the cursor support provided by the ODBC Cursor Library will be used. In general, this flag should be off (set to 0) unless you have a specific reason for not using the ODBC Cursor Library.

StreamPrefetch (InterSystems IRIS only)

Sets the stream prefetch size. All streams smaller than or equal to the prefetch size will be fetched in entirety as part of the original result set, thus eliminating the need for additional server round trips. Any streams larger than prefetch size will not be prefetched. Stream prefetch is off by default. If it is on, setting the size to 0 will turn it off again.

Stream prefetching is only advantageous with short streams. Values up to 100k are accepted. Smaller values such as 10k or 30k are advised. Values outside of the permitted range will not be accepted and the stream prefetch size will remain whatever it was before.

Stream prefetch may only be used with non-lob data retrieval methods (`getAsciiStream`, `getBinaryStream` and `getCharacterStream`). An attempt to access prefetched data with lob methods (`getBlob` and `getClob`) will result in an exception.

Trace

Specifies whether the driver manager performs logging ("Trace = on") or not ("Trace = off") By default, logging is off (see “[ODBC Logging on UNIX®](#)” for more information).

TraceFile

If logging is enabled by the Trace option, this specifies the location of the driver manager log file. For example

```
"TraceFile = iodbctrace.log"
```

UID (or User Name)

Specifies the username for logging in to the DSN. The InterSystems default value is `_SYSTEM` (not case-sensitive).

Unicode SQLTypes (or UnicodeSQLTypes)

If set to 1, this option turns on reporting of a Unicode SQL type (`SQL_WVARCHAR (-9) SQLType`) for string data.

This functionality is only relevant when working with a multibyte character set. It allows some Microsoft applications to allocate the properly sized buffers to hold multibyte data. If an application encounters a “SQL data type out of range” error from the Microsoft Driver Manager using **SQLBindParameter**, it can be caused by having selected this option.